# Лекция 12: тестирование свойств

Функциональное программирование на Haskell

---

Алексей Романов

31 октября 2026 г.

МИЭТ

# Привычный подход к тестированию

- Как обычно мы тестируем написанные функции?

# Привычный подход к тестированию

- Как обычно мы тестируем написанные функции?
- Вычисляем функцию на каких-то конкретных аргументах и сравниваем результат с ожидаемым.
- Плюсы:

# Привычный подход к тестированию

- Как обычно мы тестируем написанные функции?
- Вычисляем функцию на каких-то конкретных аргументах и сравниваем результат с ожидаемым.
- Плюсы:
  - Тесты легко понять.
  - Можно сделать даже без помощи библиотек.
  - Но при этом библиотеки для написания таких тестов есть практически для всех языков.
  - Чистую функцию удобно тестировать: результат зависит только от аргументов.

# Привычный подход к тестированию

- Как обычно мы тестируем написанные функции?
- Вычисляем функцию на каких-то конкретных аргументах и сравниваем результат с ожидаемым.
- Плюсы:
  - Тесты легко понять.
  - Можно сделать даже без помощи библиотек.
  - Но при этом библиотеки для написания таких тестов есть практически для всех языков.
  - Чистую функцию удобно тестировать: результат зависит только от аргументов.
- TDD (Test Driven Development)

# Привычный подход к тестированию

- Как обычно мы тестируем написанные функции?
- Вычисляем функцию на каких-то конкретных аргументах и сравниваем результат с ожидаемым.
- Плюсы:
  - Тесты легко понять.
  - Можно сделать даже без помощи библиотек.
  - Но при этом библиотеки для написания таких тестов есть практически для всех языков.
  - Чистую функцию удобно тестировать: результат зависит только от аргументов.
- TDD (Test Driven Development) — сначала тесты, потом реализация.

# Недостатки этого подхода

- А какие у этого подхода минусы?

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё.

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё. Разве что если функция на `Bool`...

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё. Разве что если функция на `Bool`...
- Нужно искать пограничные случаи (т.е. те, в которых ошибка более вероятна) самому.
  Хотя есть часто встречающиеся:

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё. Разве что если функция на `Bool`...
- Нужно искать пограничные случаи (т.е. те, в которых ошибка более вероятна) самому. Хотя есть часто встречающиеся: `0`, пустой список, `(minBound :: Int)`...

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё. Разве что если функция на `Bool`...
- Нужно искать пограничные случаи (т.е. те, в которых ошибка более вероятна) самому.
  Хотя есть часто встречающиеся: `0`, пустой список, `(minBound :: Int)`...
- Для каких-то аргументов мы можем не знать правильного результата (и не иметь независимого от тестируемой функции способа его найти).

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё. Разве что если функция на `Bool`...
- Нужно искать пограничные случаи (т.е. те, в которых ошибка более вероятна) самому.
  Хотя есть часто встречающиеся: `0`, пустой список, `(minBound :: Int)`...
- Для каких-то аргументов мы можем не знать правильного результата (и не иметь независимого от тестируемой функции способа его найти).
- Есть ситуации, когда программу или функцию в ней намеренно пытаются сломать.

# Недостатки этого подхода

- А какие у этого подхода минусы?
- Сколько нужно тестов? Всегда можно добавить ещё. Разве что если функция на `Bool`...
- Нужно искать пограничные случаи (т.е. те, в которых ошибка более вероятна) самому.
  Хотя есть часто встречающиеся: `0`, пустой список, `(minBound :: Int)`...
- Для каких-то аргументов мы можем не знать правильного результата (и не иметь независимого от тестируемой функции способа его найти).
- Есть ситуации, когда программу или функцию в ней намеренно пытаются сломать. Например, злобный преподаватель.

# Свойства вместо примеров

- Тестирование свойств (property-based testing) помогает находить ошибки на множестве автоматически сгенерированных примеров.
- Основная идея: вместо результата на конкретных аргументах мы описываем свойства, которые должны выполняться *для всех аргументов* подходящего типа.

# Свойства вместо примеров

- Тестирование свойств (property-based testing) помогает находить ошибки на множестве автоматически сгенерированных примеров.
- Основная идея: вместо результата на конкретных аргументах мы описываем свойства, которые должны выполняться *для всех аргументов* подходящего типа. Или не совсем для всех, но об этом потом.

# Свойства вместо примеров

- Тестирование свойств (property-based testing) помогает находить ошибки на множестве автоматически сгенерированных примеров.
- Основная идея: вместо результата на конкретных аргументах мы описываем свойства, которые должны выполняться *для всех аргументов* подходящего типа. Или не совсем для всех, но об этом потом.
- Эти свойства проверяем на случайных аргументах (скажем, 100 за раз по умолчанию).

# Что означает успешная проверка?

- В процессе разработки тесты запускаются далеко не один раз, так что каждое свойство будет проверено в сотнях или тысячах случаев.

# Что означает успешная проверка?

- В процессе разработки тесты запускаются далеко не один раз, так что каждое свойство будет проверено в сотнях или тысячах случаев.
- Успешная проверка не доказывает свойство: результат зависит от выбора генераторов и числа тестов.

# Что означает успешная проверка?

- В процессе разработки тесты запускаются далеко не один раз, так что каждое свойство будет проверено в сотнях или тысячах случаев.
- Успешная проверка не доказывает свойство: результат зависит от выбора генераторов и числа тестов.
- Важные пограничные случаи всё равно нужно учитывать при построении генераторов.

# Простейший пример

- Будем использовать библиотеку hedgehog:

```
import Hedgehog
import qualified Hedgehog.Gen as Gen
import qualified Hedgehog.Range as Range

prop_square :: Property
prop_square = property $ do
  (x :: Integer) <- forAll $ Gen.integral $
    Range.linearFrom 0 (-10^100) (10^100)
  assert (x * x >= 0)

ghci> check prop_square
True
```

- Квадраты оказались неотрицательны для 100 случайных значений в данном диапазоне.

# Диапазоны значений

- `Hedgehog.Range` задаёт границы значений и цель уменьшения.
- `Range.linear 0 100`: границы растут от `(0, 0)` до `(0, 100)` вместе с размером примера (от 0 до 99).
- `Range.constant 0 100`: границы всегда `(0, 100)`, независимо от размера.
- `Range.linearFrom 0 (-100) 100`: границы расширяются от нуля в обе стороны.
- `Range.constantBounded`: весь диапазон ограниченного типа, например `Int`.
- Во всех этих примерах цель уменьшения — ноль.

# А если использовать Int?

- Верно ли то же свойство для `Int`?

```
prop_squareInt :: Property
prop_squareInt = property $ do
  x <- forAll $ Gen.int Range.constantBounded
  diff (x * x) (>=) 0
ghci> check prop_squareInt
```

# А если использовать Int?

- Верно ли то же свойство для `Int`?

```
prop_squareInt :: Property
prop_squareInt = property $ do
  x <- forAll $ Gen.int Range.constantBounded
  diff (x * x) (>=) 0
ghci> check prop_squareInt
```

Фрагмент отчёта (64-битный `Int`):

```
    after 2 tests and 27 shrinks.
6 |   x <- forAll $ Gen.int Range.constantBounded
  |   | 3013262616387239886
7 |   diff (x * x) (>=) 0
  |   ^^^^^^^^^^^^^^^^^^^
  |   | ━━━ Failed (- lhs) (+ rhs) ━━━
  |   | - -7464790805971924540
  |   | + 0
False
```

# Переполнение и минимизация

- `diff a rel b` проверяет `rel a b`. При ошибке показывает оба значения: `-` обозначает левое, `+` — правое.
- Контрпример уменьшен, но не до минимума. Для 64-битного `Int` есть, например:

```
ghci> let x = 3037000500 :: Int
ghci> x * x
```

# Переполнение и минимизация

- `diff a rel b` проверяет `rel a b`. При ошибке показывает оба значения: `-` обозначает левое, `+` — правое.
- Контрпример уменьшен, но не до минимума. Для 64-битного `Int` есть, например:

```
ghci> let x = 3037000500 :: Int
ghci> x * x
-9223372036709301616
```

- Из-за переполнения свойство `x * x >= 0` может менять истинность при уменьшении `x`. Shrinking не гарантирует самого маленького контрпримера.

# Ещё один пример

- Первый элемент списка равен последнему в развёрнутом списке?

```
prop_head :: Property
prop_head = property $ do
  xs <- forAll $ Gen.list (Range.linear 0 100) $
    Gen.int (Range.linearFrom 0 (-100) 100)
  head xs === last (reverse xs)
ghci> check prop_head
```

# Ещё один пример

- Первый элемент списка равен последнему в развёрнутом списке?

```
prop_head :: Property
prop_head = property $ do
  xs <- forAll $ Gen.list (Range.linear 0 100) $
    Gen.int (Range.linearFrom 0 (-100) 100)
  head xs === last (reverse xs)
ghci> check prop_head
Prelude.head: empty list
False
```

  Вывод сокращён; `head` даёт предупреждение `x-partial`.

- Как исправить?

# Ещё один пример

- Первый элемент списка равен последнему в развёрнутом списке?

```
prop_head :: Property
prop_head = property $ do
  xs <- forAll $ Gen.list (Range.linear 0 100) $
    Gen.int (Range.linearFrom 0 (-100) 100)
  head xs === last (reverse xs)
ghci> check prop_head
Prelude.head: empty list
False
```

  Вывод сокращён; `head` даёт предупреждение `x-partial`.

- Как исправить? Отбросим пустые списки перед вызовом `head`:

```
if null xs then discard else success
```

  Или генерировать непустые списки: `Range.linear 1 100`.

# Основные типы и функции

- `Gen a`: генератор случайных значений типа `a`.
- `Property`: свойство, которое можно проверить.
- Тело свойства — в монаде `PropertyT IO` (типы упрощены):

```
property :: PropertyT IO () -> Property
forAll :: Show a => Gen a -> PropertyT IO a
assert :: Bool -> PropertyT IO ()
(===) :: (Eq a, Show a) =>
         a -> a -> PropertyT IO ()
check :: Property -> IO Bool
```

- `forAll` генерирует аргумент; `assert` проверяет условие; `===` сравнивает значения.
- `discard` пропускает пример. Слишком много пропусков — проблема с генератором.

# Генераторы значений

- Примеры базовых генераторов:

```
Gen.int :: Range Int -> Gen Int
Gen.element :: [a] -> Gen a
```

- Из простых генераторов можно строить более сложные, пользуясь тем, что `Gen` — монада. Например:

```
genPair :: Gen a -> Gen b -> Gen (a, b)
genPair ga gb =
```

# Генераторы значений

- Примеры базовых генераторов:

```
Gen.int :: Range Int -> Gen Int
Gen.element :: [a] -> Gen a
```

- Из простых генераторов можно строить более сложные, пользуясь тем, что `Gen` — монада. Например:

```
genPair :: Gen a -> Gen b -> Gen (a, b)
genPair ga gb = liftA2 (,) ga gb

genMaybe :: Gen a -> Gen (Maybe a)
genMaybe ga = do is_just <- Gen.bool
                 if is_just
                   then
```

# Генераторы значений

- Примеры базовых генераторов:

```
Gen.int :: Range Int -> Gen Int
Gen.element :: [a] -> Gen a
```

- Из простых генераторов можно строить более сложные, пользуясь тем, что `Gen` — монада. Например:

```
genPair :: Gen a -> Gen b -> Gen (a, b)
genPair ga gb = liftA2 (,) ga gb

genMaybe :: Gen a -> Gen (Maybe a)
genMaybe ga = do is_just <- Gen.bool
                 if is_just
                   then fmap Just ga
                   else
```

# Генераторы значений

- Примеры базовых генераторов:

```
Gen.int :: Range Int -> Gen Int
Gen.element :: [a] -> Gen a
```

- Из простых генераторов можно строить более сложные, пользуясь тем, что `Gen` — монада. Например:

```
genPair :: Gen a -> Gen b -> Gen (a, b)
genPair ga gb = liftA2 (,) ga gb

genMaybe :: Gen a -> Gen (Maybe a)
genMaybe ga = do is_just <- Gen.bool
                 if is_just
                   then fmap Just ga
                   else pure Nothing
```

# Композиция и уменьшение

- Вариант через `do` генерирует пары так же:

```haskell
genPairM :: Gen a -> Gen b -> Gen (a, b)
genPairM ga gb = do
  x <- ga
  y <- gb
  pure (x, y)
```

# Композиция и уменьшение

- Вариант через `do` генерирует пары так же:

```haskell
genPairM :: Gen a -> Gen b -> Gen (a, b)
genPairM ga gb = do
  x <- ga
  y <- gb
  pure (x, y)
```

- Начав уменьшать `y` в контрпримере, Hedgehog уже не возвращается к `x`. С `liftA2` можно чередовать уменьшение компонент.
- То же ограничение — у нескольких `forAll` в `do`.
- В falsify этого ограничения нет: он уменьшает внутренние случайные решения.

# Распределение тестовых примеров

- `classify` показывает доли случаев, а `cover` требует минимальную долю:

```
{-# LANGUAGE OverloadedStrings #-}
prop_reverseCoverage :: Property
prop_reverseCoverage = property $ do
  xs <- forAll $ Gen.list
    (Range.constant 0 10) Gen.bool
  classify "empty" (null xs)
  cover 30 "length >= 3" (length xs >= 3)
  reverse (reverse xs) === xs
ghci> check prop_reverseCoverage
```

# Распределение тестовых примеров

- `classify` показывает доли случаев, а `cover` требует минимальную долю:

```
{-# LANGUAGE OverloadedStrings #-}
prop_reverseCoverage :: Property
prop_reverseCoverage = property $ do
  xs <- forAll $ Gen.list
    (Range.constant 0 10) Gen.bool
  classify "empty" (null xs)
  cover 30 "length >= 3" (length xs >= 3)
  reverse (reverse xs) === xs
ghci> check prop_reverseCoverage
```

Фрагмент сводки одного запуска (без диаграмм):

```
empty          5%
length >= 3 82%
```

- С `pure []` свойство верно, но проверка покрытия провалится.

# Генераторы значений

- Для рекурсивных типов наивный генератор:

```
import Control.Applicative (liftA3)
data Tree a = Empty | Node a (Tree a) (Tree a)
  deriving (Eq, Show)

treeOf :: Gen a -> Gen (Tree a)
treeOf g = Gen.choice [pure Empty,
              liftA3 Node g (treeOf g) (treeOf g)]
```

  но это определение плохое: оно даёт очень маленькие деревья (почему?).

- Правильнее:

```
treeOf g = Gen.recursive Gen.choice
  [pure Empty]
  [liftA3 Node g (treeOf g) (treeOf g)]
```

  `Gen.recursive` уменьшает размер при рекурсии и исключает её при малом размере.

# Почему деревья маленькие?

- В первом варианте `treeOf` каждый узел с вероятностью $1/2$ — лист, иначе у него два потомка.
- Это ветвящийся процесс (Гальтона-Ватсона) со средним числом потомков $2 \cdot 1/2 = 1$ — *критический* случай.

# Почему деревья маленькие?

- В первом варианте `treeOf` каждый узел с вероятностью $1/2$ — лист, иначе у него два потомка.
- Это ветвящийся процесс (Гальтона-Ватсона) со средним числом потомков $2 \cdot 1/2 = 1$ — *критический* случай.
- Дерево конечно с вероятностью 1, но его *ожидаемый* размер бесконечен: почти всегда оно крошечное, а изредка генерация не завершается за разумное время.
- Отсюда и параметр размера в Hedgehog (`Gen.sized`, `Gen.resize`): размер должен убывать при рекурсии. `Gen.recursive` делает это за нас.

# Минимизация контрпримеров

- Первый найденный контрпример может быть большим.
- Библиотека ищет более простые аргументы, дающие ту же ошибку.
- В Hedgehog уменьшение (shrinking) встроено в генератор: отдельная функция обычно не нужна.
- Например, генератор непустых списков уменьшает контрпример, сохраняя непустоту.
- Это не гарантирует минимального контрпримера: качество уменьшения зависит от того, как построен генератор.
- Подробнее: Integrated vs type based shrinking.

# Воспроизводимость

- Если мы нашли контрпример (например, с десятого запуска) и изменили реализацию, нужно проверить, что этот случай исправился.
- Для этого можно добавить обычный тест с напечатанными аргументами.
- Конечно, это удобно делать только тогда, когда минимизация более-менее удалась.
- Кроме того, для некоторых типов результат `show x` недостаточен для точного воспроизведения `x` (придумаете пример?)
- В Hedgehog печатается команда с `recheckAt` для воспроизведения по seed и пути уменьшения. Перед её запуском в GHCi:

```
ghci> :set -XOverloadedStrings
```

# Дополнительные уменьшения для своих типов

- В `genPair ga gb` уменьшение обеих компонент уже задано генераторами `ga` и `gb`.
- Можно добавить свои варианты уменьшения с помощью `Gen.shrink`:

```
shrinkTree :: Tree a -> [Tree a]
shrinkTree Empty = []
shrinkTree (Node _ l r) = [Empty, l, r]

genTree :: Gen a -> Gen (Tree a)
genTree g = Gen.shrink shrinkTree (treeOf g)
```

- Дополнительно пробуем пустое дерево или поддерево.

# Часто встречающиеся виды свойств

- Самое тривиальное свойство — функция даёт результат, а не выкидывает исключение.
- Сверка с другой реализацией той же функции.
- Алгебраические свойства: идемпотентность, коммутативность, ассоциативность и так далее.
- Законы классов типов.

# Свойства сериализации и разбора

- Для некоторого типа значений `Value`:
  - Предположим типы:
    ```
    encode :: Value -> String
    decode :: String -> Either String Value
    ```
  - Из значения в представление и обратно:
    ```
    decode (encode x) === Right x
    ```
    Проверяем, что кодирование сохраняет значение.

# Свойства сериализации и разбора

- Для некоторого типа значений `Value`:
  - Предположим типы:
    ```
    encode :: Value -> String
    decode :: String -> Either String Value
    ```
  - Из значения в представление и обратно:
    ```
    decode (encode x) === Right x
    ```
    Проверяем, что кодирование сохраняет значение.
  - Из представления в значение и обратно:
    ```
    fmap encode (decode s) === Right s
    ```
    Требует корректной канонической записи: в той форме, которую выдаёт `encode`.
    Повторное кодирование может нормализовать запись: например, разбор `"01"` даёт число `1`, а его запись — `"1"`.

# Генераторы функций

- Для тестирования функций высшего порядка нужны случайные функции.
- Для Hedgehog это предоставляет пакет hedgehog-fn, модуль `Hedgehog.Function`:

```
import Hedgehog.Function
  (Arg, Vary, Fn, fn, apply)

fn :: (Arg a, Vary a) => Gen b -> Gen (Fn a b)
apply :: Fn a b -> a -> b
```

- `Fn a b` представляет функцию `a -> b` в виде, который можно показать и уменьшить.
- Экземпляры `Arg a` и `Vary a` нужны для генерации функций с аргументами типа `a`; для стандартных типов они уже есть.

# Свойства функций

- Проверим ассоциативность композиции для функций `Int -> Int`:

```
prop_compose :: Property
prop_compose = property $ do
  let gi = Gen.int (Range.linearFrom 0 (-100) 100)
  (fg, fh, fk, x) <- forAll $
    (,,,) <$> fn gi <*> fn gi <*> fn gi <*> gi
  let (g, h, k) = (apply fg, apply fh, apply fk)
  ((g . h) . k) x === (g . (h . k)) x
ghci> check prop_compose
```

# Свойства функций

- Проверим ассоциативность композиции для функций `Int -> Int`:

```
prop_compose :: Property
prop_compose = property $ do
  let gi = Gen.int (Range.linearFrom 0 (-100) 100)
  (fg, fh, fk, x) <- forAll $
    (,,,) <$> fn gi <*> fn gi <*> fn gi <*> gi
  let (g, h, k) = (apply fg, apply fh, apply fk)
  ((g . h) . k) x === (g . (h . k)) x
ghci> check prop_compose
True
```

- У функций нет `Eq`: сравниваем результаты на аргументе `x`. Функция `apply` извлекает обычную функцию из `Fn`.

# Закон класса как свойство

- Например, проверим ассоциативность операции `<>`:

```
prop_associative :: (Eq a, Show a, Semigroup a)
                 => Gen a -> Property
prop_associative genA = property $ do
  (x, y, z) <- forAll $
    (,,) <$> genA <*> genA <*> genA
  (x <> y) <> z === x <> (y <> z)
```

- Генератор определяет, какой экземпляр класса проверяем. Например, для списков `<>` — конкатенация:

```
ghci> check $ prop_associative $
        Gen.list (Range.linear 0 20) $
          Gen.int (Range.linearFrom 0 (-100) 100)
```

# Закон класса как свойство

- Например, проверим ассоциативность операции `<>`:

```
prop_associative :: (Eq a, Show a, Semigroup a)
                 => Gen a -> Property
prop_associative genA = property $ do
  (x, y, z) <- forAll $
    (,,) <$> genA <*> genA <*> genA
  (x <> y) <> z === x <> (y <> z)
```

- Генератор определяет, какой экземпляр класса проверяем. Например, для списков `<>` — конкатенация:

```
ghci> check $ prop_associative $
        Gen.list (Range.linear 0 20) $
          Gen.int (Range.linearFrom 0 (-100) 100)
True
```

# Тестирование законов классов

- Библиотека hedgehog-classes содержит свойства, проверяющие законы для множества классов.
- Например, для `Eq`:

```
import Hedgehog.Classes

eqLaws :: (Eq a, Show a) => Gen a -> Laws
lawsCheck :: Laws -> IO Bool

ghci> lawsCheck $ eqLaws $
        Gen.int (Range.linearFrom 0 (-100) 100)
```

# Тестирование законов классов

- Библиотека hedgehog-classes содержит свойства, проверяющие законы для множества классов.
- Например, для `Eq`:

```
import Hedgehog.Classes

eqLaws :: (Eq a, Show a) => Gen a -> Laws
lawsCheck :: Laws -> IO Bool

ghci> lawsCheck $ eqLaws $
        Gen.int (Range.linearFrom 0 (-100) 100)
```

- Для проверки законов `Functor`/`Applicative`/`Monad`/`...` передаём генератор структуры, например:

```
ghci> lawsCheck (functorLaws Gen.maybe)
```

# Свойства с IO

- Проверим запись и чтение `IORef` из `Data.IORef`:

```
import Data.IORef (newIORef, writeIORef, readIORef)

prop_ioRef :: Property
prop_ioRef = property $ do
  x <- forAll $ Gen.int Range.constantBounded
  y <- evalIO $ do
    ref <- newIORef 0
    writeIORef ref x
    readIORef ref
  y === x
ghci> check prop_ioRef
```

# Свойства с IO

- Проверим запись и чтение `IORef` из `Data.IORef`:

```
import Data.IORef (newIORef, writeIORef, readIORef)

prop_ioRef :: Property
prop_ioRef = property $ do
  x <- forAll $ Gen.int Range.constantBounded
  y <- evalIO $ do
    ref <- newIORef 0
    writeIORef ref x
    readIORef ref
  y === x
ghci> check prop_ioRef
True
```

- `evalIO` выполняет действие `IO` внутри свойства. Новый `IORef` создаётся для каждого примера, в том числе при уменьшении.

# Альтернативы: QuickCheck

- QuickCheck: класс `Arbitrary` задаёт генератор `arbitrary` и отдельную функцию уменьшения `shrink`.
- Для стандартных типов генератор уже есть; можно сразу записать свойство как функцию:

```
import Test.QuickCheck

prop_reverse :: [Int] -> Property
prop_reverse xs = reverse (reverse xs) === xs

ghci> quickCheck prop_reverse
+++ OK, passed 100 tests.
```

- Генераторы можно передавать и явно, но уменьшение задаётся отдельно.

# QuickCheck: переполнение Int

- Проверим неотрицательность квадрата:

```
prop_squareInt :: Int -> Bool
prop_squareInt x = x * x >= 0
ghci> quickCheck prop_squareInt
```

# QuickCheck: переполнение Int

- Проверим неотрицательность квадрата:

```
prop_squareInt :: Int -> Bool
prop_squareInt x = x * x >= 0
ghci> quickCheck prop_squareInt
+++ OK, passed 100 tests.
```

- Стандартный генератор `Int` начинает с малых значений. Возьмём весь диапазон и явно зададим уменьшение:

```
ghci> quickCheck $
        forAllShrink arbitraryBoundedIntegral shrink
          prop_squareInt
```

# QuickCheck: переполнение Int

- Проверим неотрицательность квадрата:

```
prop_squareInt :: Int -> Bool
prop_squareInt x = x * x >= 0
ghci> quickCheck prop_squareInt
+++ OK, passed 100 tests.
```

- Стандартный генератор `Int` начинает с малых значений. Возьмём весь диапазон и явно зададим уменьшение:

```
ghci> quickCheck $
        forAllShrink arbitraryBoundedIntegral shrink
          prop_squareInt
```

Пример вывода (64-битный `Int`):

```
*** Failed! Falsified (after 1 test and 64 shrinks):
3037000500
```

# Альтернативы: Falsify

- falsify: уменьшает внутренние случайные решения; поддерживает уменьшение через зависимые генераторы.

```haskell
{-# LANGUAGE OverloadedStrings #-}
import Test.Falsify
import Test.Falsify.Interactive (falsify)
import qualified Test.Falsify.Generator as Gen
import qualified Test.Falsify.Predicate as P
import qualified Test.Falsify.Range as Range
```

- Свойство имеет тип `Property ()`; `gen` играет роль `forAll` из Hedgehog.
- `assert` принимает специальный предикат, например `P.eq`. Оператор `.$` передаёт ему аргумент вместе с подписью для вывода ошибки.

# Falsify: пример

- Та же проверка двойного разворота списка:

```
prop_reverse :: Property ()
prop_reverse = do
  xs <- gen $ Gen.list (Range.inclusive (0, 100)) $
    Gen.int (Range.inclusive (-100, 100))
  assert $ P.eq .$ ("actual", reverse (reverse xs))
                .$ ("expected", xs)
```

```
ghci> falsify prop_reverse
Nothing
```

- `Nothing` означает, что контрпример не найден; при нарушении свойства получим `Just` с описанием ошибки.

# Falsify: переполнение Int

- `Range.uniform` охватывает весь диапазон типа и задаёт уменьшение к нулю:

```
prop_squareInt :: Property ()
prop_squareInt = do
  x <- gen $ Gen.int Range.uniform
  assert $ P.ge .$ ("x * x", x * x)
                .$ ("zero", 0)
ghci> falsify prop_squareInt
```

# Falsify: переполнение Int

- `Range.uniform` охватывает весь диапазон типа и задаёт уменьшение к нулю:

```
prop_squareInt :: Property ()
prop_squareInt = do
  x <- gen $ Gen.int Range.uniform
  assert $ P.ge .$ ("x * x", x * x)
                .$ ("zero", 0)
ghci> falsify prop_squareInt
```

- Получим `Just` с описанием ошибки. Пример сообщения для 64-битного `Int`:

```
x * x < zero
x * x: -9223372036709301616
zero : 0
```

- `P.ge` проверяет `>=`; подписи помогают понять, что именно оказалось неверным.

# Альтернативы: TinyCheck

- TinyCheck: перебирает значения от простых к сложным вместо случайной генерации. Проверки детерминированы; отдельное уменьшение не требуется.
- Пример с библиотекой запуска тестов tasty:

```haskell
import Test.Tasty (defaultMain)
import Test.Tasty.TinyCheck (testProperty)

main :: IO ()
main = defaultMain $
  testProperty "reverse . reverse == id" $
    \(xs :: [Int]) -> reverse (reverse xs) == xs
```

- Использует свой класс `Arbitrary` для перечисления примеров; экземпляры для стандартных типов уже есть.

# TinyCheck: переполнение Int

- Та же проверка со стандартным перечислением:

```haskell
main :: IO ()
main = defaultMain $
  testProperty "square" $
    \(x :: Int) -> x * x >= 0
```

# TinyCheck: переполнение Int

- Та же проверка со стандартным перечислением:

```
main :: IO ()
main = defaultMain $
  testProperty "square" $
    \(x :: Int) -> x * x >= 0
```

```
square: OK
  OK, checked 10000 cases
```

- Перебираются `0, -1, 1, -2, 2, ...`: до значений, вызывающих переполнение, ещё очень далеко.
- Добавим большие значения в перечисление явно.

# TinyCheck: добавляем большие значения

- Значения вида $2^k - 1$ чередуем со стандартными:

```
import Data.Bits (finiteBitSize)
import Data.TestCases (TestCases(..), arbitrary)
import Test.Tasty.TinyCheck (forAllShow, assert)

ints :: TestCases Int
ints = TestCases [2^k - 1 | k <- [0 .. bits - 1]]
  <> arbitrary
  where bits = finiteBitSize (0 :: Int)
main = defaultMain $ testProperty "square" $ do
  x <- forAllShow "x" ints
  assert (x * x >= 0)
```

# TinyCheck: добавляем большие значения

- Значения вида $2^k - 1$ чередуем со стандартными:

```
import Data.Bits (finiteBitSize)
import Data.TestCases (TestCases(..), arbitrary)
import Test.Tasty.TinyCheck (forAllShow, assert)

ints :: TestCases Int
ints = TestCases [2^k - 1 | k <- [0 .. bits - 1]]
  <> arbitrary
  where bits = finiteBitSize (0 :: Int)
main = defaultMain $ testProperty "square" $ do
  x <- forAllShow "x" ints
  assert (x * x >= 0)
```

Сокращённый вывод для 64-битного `Int`:

```
square: FAIL
  x: 4294967295
  falsified
```

# Дополнительное чтение

- QuickCheck in Every Language Список библиотек тестирования свойств на разных языках на апрель 2016 (от автора Hypothesis для Python).
- The Design and Use of QuickCheck
- Property testing with Hedgehog
- Integrated versus Manual Shrinking Реализация и сравнение мини-версий QuickCheck и Hedgehog.
- falsify: Hypothesis-inspired shrinking for Haskell (2023).
- Как перестать беспокоиться и начать писать тесты на основе свойств
- Finding Property Tests
- Property-Based Testing in a Screencast Editor
- Verifying Typeclass Laws in Haskell with QuickCheck
- Using Hypothesis and Pexpect to Test High School Programming Assignments